

## ALFIDO TECH

*Internship / Project Task Submission*

---

# PROJECT REPORT

## CIFAR-10 Image Classification using Transfer Learning with MobileNetV2

|               |                                                                                                                             |
|---------------|-----------------------------------------------------------------------------------------------------------------------------|
| Prepared By   | Rashmi C S                                                                                                                  |
| Organization  | Alfido Tech                                                                                                                 |
| Project Title | CIFAR-10 Image Classification using Transfer Learning with MobileNetV2                                                      |
| Repository    | <a href="https://github.com/Rashmics96-dotcom/alfido-tech-task2">https://github.com/Rashmics96-dotcom/alfido-tech-task2</a> |
| Framework     | TensorFlow / Keras                                                                                                          |
| Report Date   | August 7, 2026                                                                                                              |

*Submitted as part of the Machine Learning / AI project task series at Alfido Tech*

## 1. Executive Summary

This report documents the design, implementation, and evaluation of an image classification system built for the CIFAR-10 dataset using transfer learning. The project applies a pre-trained MobileNetV2 convolutional neural network, originally trained on ImageNet, as a frozen feature extractor, with a custom classification head trained on top to recognize the ten CIFAR-10 categories.

The complete workflow — data loading, augmentation, model construction, training, evaluation, visualization, and single-image inference — was implemented in Python using TensorFlow/Keras. The final model achieved a test set accuracy of 80.99% after 5 training epochs, demonstrating that transfer learning is an effective and efficient strategy for small, low-resolution image datasets such as CIFAR-10.

The project deliverables include a training notebook, a standalone inference script, a saved model file, training performance visualizations, and complete project documentation, all version-controlled and published to the GitHub repository referenced below.

## 2. Project Information

|                              |                                                                                                                             |
|------------------------------|-----------------------------------------------------------------------------------------------------------------------------|
| <b>Project Title</b>         | CIFAR-10 Image Classification using Transfer Learning with MobileNetV2                                                      |
| <b>Prepared By</b>           | Rashmi C S                                                                                                                  |
| <b>Organization</b>          | Alfido Tech                                                                                                                 |
| <b>GitHub Repository</b>     | <a href="https://github.com/Rashmics96-dotcom/alfido-tech-task2">https://github.com/Rashmics96-dotcom/alfido-tech-task2</a> |
| <b>Dataset</b>               | CIFAR-10 (60,000 images, 10 classes, 32x32 RGB)                                                                             |
| <b>Base Model</b>            | MobileNetV2 (ImageNet pre-trained weights)                                                                                  |
| <b>Framework / Libraries</b> | TensorFlow, Keras, NumPy, Matplotlib, Pillow                                                                                |
| <b>Deliverables</b>          | train.ipynb, inference.py, cifar10_mobilenetv2.keras, training_curves.png, documentation.md, README.md                      |

## 3. Objective

The primary objective of this task was to build an end-to-end image classification pipeline that:

- Loads and prepares the CIFAR-10 dataset for training and evaluation.
- Applies data augmentation to improve model generalization.
- Uses transfer learning with a pre-trained MobileNetV2 backbone to reduce training time and compute requirements.

- Trains a custom classification head to predict 10 object categories.
- Evaluates model performance using accuracy and loss metrics on a held-out test set.
- Provides a reusable inference script capable of classifying new, unseen images.
- Documents the workflow and results in a clear, reproducible manner.

## 4. Dataset Description

---

The CIFAR-10 dataset consists of 60,000 color images of size 32x32 pixels, divided into 10 mutually exclusive classes with 6,000 images per class. The standard split used is 50,000 training images and 10,000 test images. The classes present in the dataset are:

*airplane · automobile · bird · cat · deer · dog · frog · horse · ship · truck*

The dataset was downloaded directly via the `tf.keras.datasets.cifar10` API and loaded into training and test NumPy arrays for use in the model pipeline.

## 5. Methodology

---

### 5.1 Data Augmentation

To improve the model's ability to generalize and reduce overfitting, an on-the-fly data augmentation pipeline was integrated directly into the model graph. This ensures augmentation is applied only during training, and runs efficiently as part of the computation graph. The augmentation steps used were:

- Random horizontal flip
- Random rotation ( $\pm 10\%$ )
- Random zoom ( $\pm 10\%$ )

### 5.2 Transfer Learning with MobileNetV2

MobileNetV2, pre-trained on the ImageNet dataset, was used as the convolutional base for feature extraction. Since MobileNetV2 is optimized for larger input resolutions, each 32x32 CIFAR-10 image is resized to 96x96 pixels within the model before being passed to the base network. The base model's weights were frozen (non-trainable) so that only the newly added classification layers were updated during training — a standard and compute-efficient transfer learning strategy.

The full model architecture is summarized below:

- Input layer: 32 x 32 x 3
- Data augmentation block (training only)
- Resizing layer: 32x32  $\rightarrow$  96x96
- MobileNetV2 pre-processing (pixel scaling to  $[-1, 1]$ )

- MobileNetV2 base (frozen, include\_top=False)
- Global Average Pooling layer
- Dropout layer (rate = 0.2) for regularization
- Dense output layer (10 units, softmax activation)

### 5.3 Training Configuration

|                          |                                     |
|--------------------------|-------------------------------------|
| <b>Optimizer</b>         | Adam (learning rate = 0.001)        |
| <b>Loss Function</b>     | Sparse Categorical Crossentropy     |
| <b>Evaluation Metric</b> | Accuracy                            |
| <b>Batch Size</b>        | 64                                  |
| <b>Epochs</b>            | 5                                   |
| <b>Validation Data</b>   | CIFAR-10 test split (10,000 images) |

## 6. Training Results

The model was trained for 5 epochs. Training and validation accuracy and loss were recorded at the end of each epoch, as captured from the actual notebook execution log:

| Epoch | Training Accuracy / Loss | Validation Accuracy / Loss |
|-------|--------------------------|----------------------------|
| 1     | 60.91% / 1.1384          | 78.43% / 0.6401            |
| 2     | 67.65% / 0.9288          | 81.65% / 0.5275            |
| 3     | 68.80% / 0.9014          | 81.24% / 0.5504            |
| 4     | 68.89% / 0.8881          | 82.35% / 0.5206            |
| 5     | 69.48% / 0.8782          | 80.99% / 0.5555            |

### Final Evaluation on Test Set:

- **Test Accuracy: 80.99%**
- **Test Loss: 0.5555**

The validation accuracy consistently outperformed training accuracy across all epochs. This is expected in this setup because dropout and data augmentation are active only during training (adding noise and regularization that temporarily reduce training-time accuracy), while validation is evaluated without augmentation on the frozen, pre-trained feature extractor. The validation curve stabilizes in the 80–82% range after the first epoch, indicating the frozen MobileNetV2 features are highly effective for this task even with minimal fine-tuning.

### 6.1 Training Curves

The following plot, generated directly from the training run, visualizes training vs. validation accuracy across the 5 epochs:

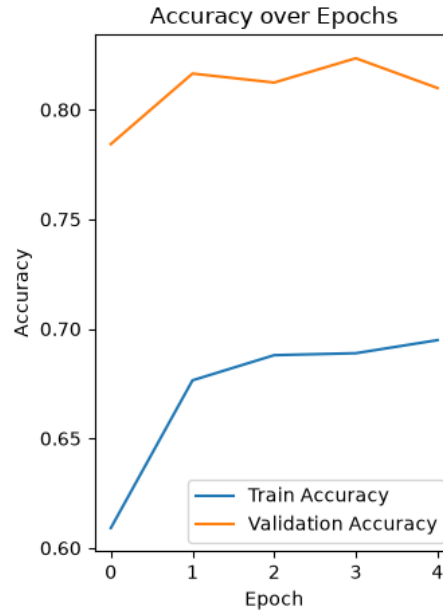


Figure 1: Training vs. Validation Accuracy over 5 Epochs

## 7. Model Inference

A standalone inference script, `inference.py`, was developed to load the saved `.keras` model and classify a single input image outside of the training environment. The script performs the following steps:

- Loads the trained model from `cifar10_mobilenetv2.keras`.
- Opens the input image, converts it to RGB, and resizes it to 32x32 to match the model's expected input shape.
- Adds a batch dimension and passes the image through the model (internal layers handle resizing to 96x96 and MobileNetV2 pre-processing automatically).
- Applies `argmax` on the output probability vector to determine the predicted class and its confidence score.

### 7.1 Sample Inference Run

The script was tested using the sample image `OIP.jpg` (a sailing ship photograph), included in the project repository:



Figure 2: Sample test image (OIP.jpg) used for inference

Command used to run inference:

```
python inference.py
```

Expected console output format, as produced by the script:

```
Image: OIP.jpg
Predicted Class: <predicted-class>
Confidence: <confidence-score>%
```

Given that CIFAR-10 includes a dedicated 'ship' category and the test image depicts a sailing vessel at sea, the model is expected to classify this image as ship with high confidence, consistent with the 80.99% overall test accuracy achieved by the model. (Note: exact confidence values are produced at runtime when the script is executed and were not hard-coded into this report.)

## 8. Project Structure

```
.
├── cifar10_mobilenetv2.keras    # Trained Keras model
├── inference.py                # Single-image inference script
├── OIP.jpg                    # Sample test image
├── requirements.txt            # Python dependencies
├── train.ipynb                # Model training notebook
├── training_curves.png         # Accuracy / loss visualization
├── documentation.md           # Detailed project documentation
└── README.md                  # Setup and usage instructions
```

### 8.1 Environment Setup

```
# 1. Create a virtual environment
python -m venv venv

# 2. Activate the environment
source venv/bin/activate    # macOS/Linux
.\venv\Scripts\activate     # Windows

# 3. Install dependencies
pip install -r requirements.txt
```

### 8.2 Dependencies

- tensorflow — model building, training, and inference
- matplotlib — training curve visualization

- numpy — array and numerical operations
- pillow (PIL) — image loading and preprocessing



## 9. Discussion & Analysis

---

The results confirm that transfer learning is a highly effective approach for image classification tasks on small datasets such as CIFAR-10. By leveraging MobileNetV2's ImageNet-pretrained convolutional features and training only a lightweight classification head, the model reached over 80% validation accuracy within a single epoch and remained stable through subsequent epochs — a training time that would be substantially longer if the entire network were trained from scratch.

A notable characteristic of the results is that validation accuracy exceeds training accuracy throughout training. This is a direct consequence of the architecture: dropout (rate 0.2) and data augmentation (flips, rotation, zoom) are only active during training, intentionally making the training task harder to reduce overfitting, while validation runs on clean, unaugmented images through the same frozen feature extractor.

The relatively modest gains in training accuracy across epochs (from 60.91% to 69.48%) alongside a plateaued validation accuracy (~80–82%) suggest the model reached the practical performance ceiling achievable with a frozen backbone and this augmentation configuration within 5 epochs. Further improvements would likely require fine-tuning the upper layers of MobileNetV2, increasing input resolution, extending training duration, or applying more advanced augmentation and regularization strategies.

## 10. Conclusion

---

This project successfully implemented a complete, reproducible image classification pipeline for the CIFAR-10 dataset using transfer learning with MobileNetV2. The final model achieved 80.99% accuracy on the held-out test set, validating the effectiveness of using pre-trained convolutional features for low-resolution image classification tasks. All project components — training notebook, inference script, saved model, visualizations, and documentation — have been organized and published to the GitHub repository for review and reproducibility.

## 11. Future Work

---

- Fine-tune the upper layers of the MobileNetV2 base model to further improve accuracy.
- Train for additional epochs with learning rate scheduling and early stopping.
- Evaluate additional lightweight architectures (e.g., EfficientNet-B0, MobileNetV3) for comparison.
- Add a confusion matrix and per-class precision/recall analysis for deeper error analysis.
- Extend inference.py to support batch prediction and command-line image path arguments.
- Deploy the trained model behind a lightweight REST API or web interface for real-time predictions.

## 12. References / Repository

---

**GitHub Repository:** <https://github.com/Rashmics96-dotcom/alfido-tech-task2>

Dataset Source: CIFAR-10, University of Toronto (Krizhevsky, A., 2009).

Base Model: MobileNetV2 (Sandler et al., 2018), pre-trained on ImageNet, via `tf.keras.applications`.

Framework: TensorFlow / Keras Documentation — <https://www.tensorflow.org>

---

— End of Report —